agent-runtime

Deep Dive: an event-sourced runtime for tool-using LLM agents

Explainer generated for the project owner

A beginner-safe, exhaustive technical walkthrough of the actual codebase

How to read this document

This document explains **exactly** what is implemented in the `agent-runtime` project folder, as of the code and tests present at the time of writing. Every claim traces to a specific file. Technical terms are defined the first time they appear, in a boxed callout like this one. Nothing here is invented: where the project's own README states an honest limitation (e.g. "no live API calls were made"), this document repeats it rather than smoothing it over.

Contents

1 What this project is

agent-runtime is a Python library and command-line tool for running *LLM agents* — programs that use a large language model (LLM) to decide, in a loop, which actions to take next — with the operational machinery that a bare "call the model, run a tool, repeat" loop leaves out: a durable record of every run, the ability to replay a run exactly, safe recovery from a crash mid-run, a human approval step before anything with a side effect happens, deliberately injected failures to prove the system degrades safely, and a suite of scripted regression tests that catch behavior changes when the underlying model changes.

Term: LLM agent

A program where a large language model is asked, in a loop, "given what has happened so far, what should happen next?" The model's answer is either a final text answer, or a request to call one or more *tools* (functions the program exposes, such as "read this file" or "send this HTTP request"). The program executes the requested tool(s), feeds the result back to the model, and asks again. This project implements exactly that loop in `src/agent_runtime/runtime.py`.

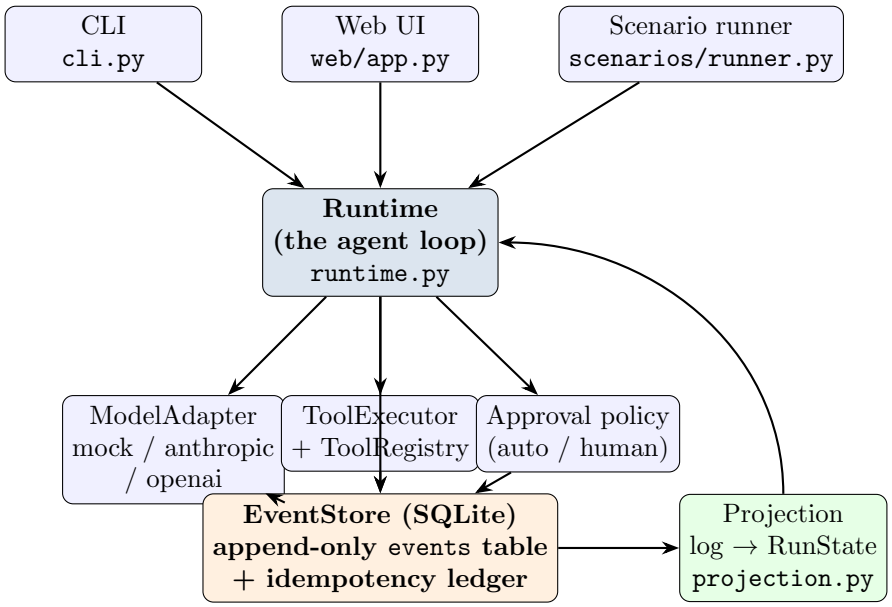
Why it exists (the project's own framing)

The project's README states its thesis directly: the agent loop itself is a small amount of code, but what makes an agent *deployable* is everything around it — knowing exactly what happened in a specific run, being able to reproduce that run, knowing whether the agent took an action (e.g. emailed a customer) before a human approved it, and knowing whether a new model version still passes the flows that used to work. The project's design response to all of this is a single idea, applied consistently: **every run is an append-only log of events, and every feature is a function computed from that log.**

Term: Event sourcing

A design pattern where, instead of storing "the current state" of something and overwriting it as things change, you store every state-changing event as an immutable, ordered record, and compute "current state" on demand by replaying those events from the start. Because the log never loses information, you can always recompute the past, replay it, or audit it. This project's central and only source of truth is such a log, one table row per event.

2 Architecture at a glance



Every arrow into the SQLite store is an *append*: the codebase contains no UPDATE or DELETE against the `events` table (confirmed by reading `store.py` in full — the schema and every SQL statement in that file are `INSERT` or `SELECT`). Every arrow out of the store goes through `projection.py`, which is the *only* module that interprets what a sequence of events means. The CLI, the web UI, the replay comparator and the scenario-test assertions all call the same `project()` function, so they cannot disagree about the state of a run.

Term: SQLite

A small, file-based relational database engine that needs no separate server process — the whole database is one file on disk. This project uses it as the single persistent store for every run's event log.

3 The event log: the one idea everything is built on

3.1 The `events` table

From `store.py`, the schema actually created is:

```
CREATE TABLE events (
    run_id TEXT NOT NULL,
    seq INTEGER NOT NULL,
    type TEXT NOT NULL,
    ts REAL NOT NULL,
    payload TEXT NOT NULL, -- JSON blob
    PRIMARY KEY (run_id, seq)
);
CREATE TABLE executions ( -- the idempotency ledger, see Sec. 6
    idem_key TEXT PRIMARY KEY,
    run_id TEXT NOT NULL,
    result TEXT NOT NULL,
    ts REAL NOT NULL
);
```

Term: Primary key / append-only

A *primary key* is a column (or combination of columns) that must be unique per row and is used to look rows up quickly. Here the primary key is `(run_id, seq)`: for a given run, each sequence number can be claimed only once. `EventStore.append()` reads `MAX(seq)+1` for the run, tries to insert, and retries (up to 5 times) if another writer raced it to the same number — this is what makes two processes appending to the same run safe without needing a lock: SQLite's uniqueness constraint on the primary key rejects the loser, who then just re-reads the new max and tries again.

3.2 Event types

`events.py` defines twelve event type constants (grouped here in the order a typical successful run emits them):

Event type	Payload highlights (from `projection.py` / `runtime.py`)
`run_created`	agent name, the user's request text, and free-form `meta` (workspace path, which adapter, scenario file if any)
`model_requested`	a call number, and a *digest* (short hash) and count of the messages about to be sent — not the messages themselves
`model_responded`	the parsed turn (text and/or tool calls), token usage, model name; or `malformed: true` with the raw unparseable text
`model_error`	a transport/provider failure for one attempt (e.g. simulated HTTP 500)
`tool_requested`	call id, tool name, arguments, and whether the tool is a side-effecting one
`tool_approved` / `tool_denied`	who approved/denied, and the deny reason if any
`tool_executed` / `tool_failed`	the tool's result or error, and how many attempts it took
`checkpoint`	how many model calls and resolved tool calls have happened so far (a periodic marker, not needed for correctness — see Sec. 4)
`run_finished` / `run_failed`	the model's final answer text, or a machine-readable failure cause string

Term: Digest / hash

A short, fixed-length string computed from a larger piece of data such that the same input always produces the same digest, and different inputs almost never produce the same one. This project uses SHA-256 (a standard cryptographic hash function) truncated to 16 hex characters purely as a fingerprint of the outgoing message list — it is not meant to be secret, just a compact "was this the same request" marker (`runtime._digest`).

3.3 Canonical encoding, for exact comparison

Every `Event` has a `canonical()` method that JSON-encodes it with sorted keys and no extra whitespace:

```
json.dumps(body, sort_keys=True, separators=(",", ":"))
```

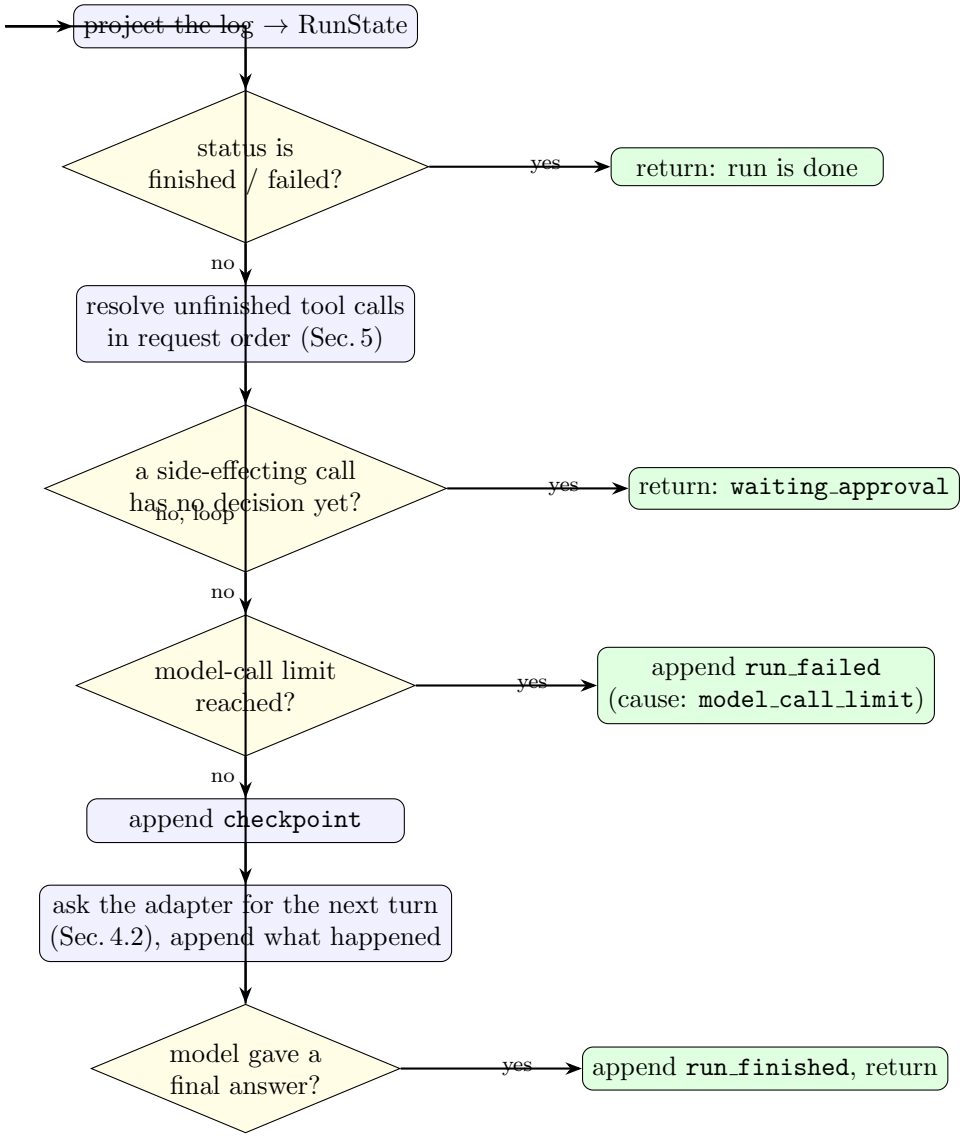
Two events that mean the same thing always produce the exact same string. This is what lets *replay* (Sec. 7) claim "byte-identical" rather than "looks about right."

4 The agent loop: `Runtime.drive()`

`runtime.py`'s `Runtime` class is the heart of the project. Its `drive(run_id)` method is, in the project's own words, "a fixed-point iteration over the log": it keeps re-deriving what to do next purely from the current event log, until the run reaches a terminal state.

Term: Fixed-point iteration

A loop that repeatedly applies the same rule to its own output until the output stops changing (or a stated exit condition is hit). Here, "derive the current state from the log, decide the single next thing to do, append it, repeat" is applied identically whether this is the very first iteration of a brand-new run, the continuation of a run just approved by a human, or the resumption of a run that crashed a minute ago. There is exactly one code path, not three.



4.1 Resolving tool calls (step "resolve unfinished tool calls")

`RunState.unfinished_calls()` returns every requested tool call that is still `requested` or `approved` (i.e. not yet executed/denied/failed), in the order they were requested. For each:

- If it is **not** a side-effecting call, or it was already approved, it executes immediately (Sec. 6).
- If it **is** side-effecting and undecided, the *approval policy* (a plain function taking the call and returning approve/deny/`None`) is consulted. `None` means "ask a human" — the loop returns `waiting_approval` right there, without touching later calls.

Term: Side effect

An action with a lasting, observable consequence outside the running program — writing a file, updating a spreadsheet, adding a calendar entry, drafting an email. Contrast with a *read-only* action (reading a file, listing a directory), which can be safely retried or re-run because it changes nothing. Every tool in this project declares a boolean `side_effect` flag (`tools/registry.py`); only side-effecting tools are gated behind human approval.

4.2 Asking the model (step "ask the adapter")

`_model_step()` rebuilds the conversation from the log via `build_messages()` (Sec. 5), appends `model_requested`, then calls `adapter.complete(messages, tool_specs)`. Three outcomes are handled:

1. **Transport/provider failure** (`AdapterError`): appends `model_error` and, if retries remain (default 2), sleeps `0.5s` $\times 2^{\texttt{attempt}}$ (exponential backoff) before retrying. If retries are exhausted, appends `run_failed` with cause `adapter_error: retries exhausted`.
2. **Unparseable output** (`MalformedOutputError`): appends `model_responded` with `malformed: true` and the raw text. If this is more than `max_malformed` (default 2) malformed responses total for the run, the run fails with cause `malformed_model_output: ...`; otherwise the loop returns `False` (not done) and the next iteration's `build_messages()` will include a corrective message telling the model to try again (Sec. 5).
3. **A parsed turn**: appended as `model_responded`. If the turn has no tool calls (`ModelTurn.is_final`), it is the final answer and `run_finished` is appended. Otherwise, every requested tool call becomes a `tool_requested` event, tagging each with whether it is side-effecting (looked up from the tool registry).

Term: Exponential backoff

A retry strategy where each successive retry waits longer than the last (here, doubling: 0.5s, 1s, 2s, ...), so a struggling remote service is not immediately hit again at full speed.

4.3 Checkpoints

Before each model call (except the very first), the loop appends a `checkpoint` event recording how many model calls and resolved tool calls have happened. Read literally in the code, a checkpoint is *not* required for the loop's correctness — everything can already be recomputed from the rest of the log — it exists as a cheap, explicit marker of progress useful for a human skimming `agentrun runs show`.

5 Rebuilding a conversation from the log

`projection.build_messages(state, system_prompt)` in `projection.py` turns the event log into the list of messages a model API expects: a system prompt, the user's original request, then one message per subsequent turn or observation — walking the log in order and translating:

Log event	Message produced
`model_responded` (normal)	an `assistant` message: the turn's text, plus `tool_calls` if any
`model_responded` (malformed)	a `user` message: "Your previous output could not be parsed. Reply with either tool calls or a final answer." — this is the corrective nudge mentioned in Sec. 4.2
`tool_executed`	a `tool` message carrying the result
`tool_failed`	a `tool` message: `{"error": ...}`
`tool_denied`	a `tool` message: `{"denied": true, "reason": ...}` — this is literally how "the operator said no" becomes something the model can react to

Because this is a pure function of the log, a run resumed after a crash sends the model *exactly* the same messages an uninterrupted run would have sent at that point — there is no separate "resume state" to keep in sync.

Each concrete adapter (Sec. 8) then translates this provider-neutral list into the wire format its specific API expects (Anthropic content blocks, or OpenAI chat messages with a `tool_call_id`).

6 Idempotency and the crash window

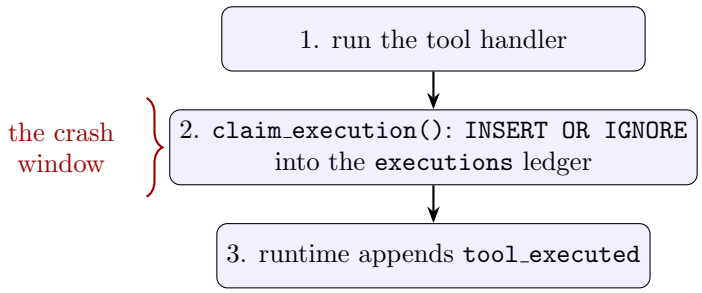
Term: Idempotency

An operation is *idempotent* if doing it twice has the same effect as doing it once. Re-reading a file is naturally idempotent. Sending a reminder email is not — sending it twice annoys the customer twice. This project makes tool execution idempotent *per run* by remembering that it already ran, so a retried/resumed run does not repeat it.

`tools/executor.py`'s `ToolExecutor.execute()` derives a key from `(run_id, seq of the tool_requested event)`:

```
def idempotency_key(run_id, requested_seq):
    return hashlib.sha256(f"{run_id}:{requested_seq}".encode()).hexdigest()[:32]
```

Both inputs are identical whether this is the first attempt or a resume after crash, so the key is stable. The write order is deliberate:



If the process dies between step 2 and step 3 — the worst possible instant, after the real-world side effect already happened but before the log says so — a subsequent resume calls `execute()` again with the *same* key, finds a row already in the `executions` ledger (via `get_execution()`), and returns that recorded result immediately without touching the handler again. The runtime then appends `tool_executed` with an extra `replayed: true` flag, so the log itself records that this particular execution didn't actually re-run. `tests/test_resume.py` exercises exactly this crash point (via a small executor wrapper that

raises *after* the handler has run but before the ledger write would normally be reached) and asserts a side-effect counter — e.g. number of `.eml` draft files written — stays at 1.

The README calls out one honest limitation here: because `concurrent.futures` timeouts (Sec. 9) abandon a worker thread rather than kill it, a genuinely hung tool leaks a thread per retried attempt; this is documented as a known gap, not silently hidden.

7 Deterministic replay

Term: Replay

Re-executing a program's logic using only what was recorded about a previous execution, with no live external calls, and checking that the result matches what was recorded. This project's replay does not call any model, run any real tool handler, or touch the network; every input the loop would normally fetch live (the next model turn, a tool's outcome, an approval decision, even the wall-clock time) is instead served from the four "recorded stand-ins" below.

`replay.py`'s `replay_run()` builds four objects, each substituting for a real dependency, all reading from the same source event list:

Stand-in	Replaces	Replays from
`_RecordedClock`	`time.time`	the exact `ts` of each original event, in order
`_RecordedAdapter`	the live `ModelAdapter`	each `model_error` or `model_responded` event, in order, re-raising errors/-malformed outputs exactly as before
`_RecordedExecutor`	`ToolExecutor`	the `tool_executed`/`tool_failed` outcome recorded for each `tool_requested`'s sequence number
`_RecordedApprovals`	the live approval policy	the `tool_approved`/`tool_denied` decision recorded per call id

The real, unmodified `Runtime` then drives the loop exactly as normal, writing to a fresh run id (`<original>-replay-<random>`) in the *same* database file. Afterwards, `canonical_log()` (Sec. 3.3) encodes both the original and the replayed event lists, and they are compared. If they differ, the code walks both lists in lock-step and reports the sequence number of the first event whose canonical form differs — so a divergence is reported as "first differing event N," not just "it's different somewhere."

`agentrun replay <run-id> --until <seq>` truncates the source log to events up to `seq` before replaying, giving a snapshot of what the run's state looked like at that exact point — useful for stepping through a failure. Truncating the log can leave the replayed loop asking for a model turn or clock tick that was never recorded; rather than special-casing this, the recorded stand-ins simply raise when exhausted, and `replay_run()` catches that `RuntimeError` and treats whatever was appended up to then as the answer.

This doubles as a regression tripwire for the runtime's own code: if a future change to the loop alters the exact sequence of events it produces, replaying an *old, previously-recorded* run will now diverge from its own recorded log and say exactly where.

8 Failure injection

Term: Fault injection

Deliberately making a system behave as if something went wrong (a timeout, a malformed response, a server error) in a controlled test, to verify the system handles that failure the way it's supposed to, instead of only ever testing the happy path.

`faults.py` defines `FaultPlan`, a plain data class (loadable straight from a scenario's YAML `faults:` block) with two kinds of entries:

```
faults:
  model:
    - {at_call: 2, kind: malformed} # or kind: adapter_500
  tools:
    read_file:
      - {at_attempt: 1, kind: exception, message: "disk error"}
      - {at_attempt: 2, kind: timeout}
```

`model_fault(call_number)` is consulted by `MockModelAdapter.complete()` on every invocation (`at_call` counts *every* adapter call, including ones that themselves fail); `tool_fault(tool_name, attempt)` is the executor's `fault_hook`, consulted before each execution attempt of a given tool (`at_attempt` counts attempts for that specific tool). Both let a scenario exercise "fails once, then a clean retry succeeds" without needing a real failing service.

The runtime's documented contract under faults: adapter errors retry with backoff then fail the run with a recorded cause; malformed output earns a corrective observation (twice), then fails the run; tool failures surface to the model as ordinary error observations it can react to. Nothing in the codebase raises an unhandled exception out of `drive()` in these paths — `tests/test_faults.py` is the suite that pins this contract down.

9 Cost and latency accounting

Every `model_responded` event carries a `usage` object: input/output token counts, latency in milliseconds, and a `simulated` boolean. `accounting.py`'s `account(events)` folds these into per-run totals using a small, hardcoded USD-per-million-token price table:

Model	Input $/1M tok	Output $/1M tok
`mock-1`	3.00	15.00
`claude-sonnet-4-5`	3.00	15.00
`gpt-4o`	2.50	10.00
(anything else)	3.00 (default)	15.00 (default)

The mock adapter (Sec. 8.1) fabricates plausible-looking but fake usage numbers deterministically from message sizes, and `RunCosts.simulated` is set `True` whenever any usage in the run came from it; the CLI and web UI both print "(simulated)" next to any such cost so it is never mistaken for a real bill. The README flags this price table as hardcoded and something that "will drift" — an explicit, honest limitation rather than a claim of live-accurate pricing.

10 Tools: the registry, the executor, and the built-ins

10.1 Typed tool declarations

Term: JSON Schema

A specification language for describing the shape of JSON data — which fields exist, their types, which are required. Tool arguments are declared this way so that badly-formed arguments (wrong type, missing required field) can be rejected before a handler ever runs, and so the same declaration can be handed to an LLM API to tell it exactly what a tool expects.

`tools/registry.py`'s `ToolSpec` dataclass captures, per tool: its `name`, `description`, a JSON-schema `parameters` definition, the Python `handler` function, the `side_effect` flag, a `timeout` (seconds), a `retries` count and a `backoff` base. `ToolRegistry` holds a set of specs, validates each schema at registration time (`jsonschema.Draft202012Validator.check_schema`), and exposes `subset(names)` — used to restrict a run to only the tools its agent definition (Sec. 11) allows.

10.2 Execution: timeout, retries, sandboxing

`tools/executor.py`'s `ToolExecutor.execute()`:

1. Checks the idempotency ledger first (Sec. 6) — if this exact call already ran, return the recorded result immediately.
2. Validates arguments against the tool's JSON schema; a validation failure is an immediate, non-retried failure (`invalid arguments: ...`).
3. Runs the handler in a `ThreadPoolExecutor` with one worker, enforcing the tool's `timeout` via `future.result(timeout=...)`.
4. Distinguishes two kinds of failure:
   - `ToolError` (and its subclass `ToolTimeout`) — an *expected*, deterministic failure (bad path, disallowed host, missing file, cell out of range). `ToolTimeout` still consumes a retry attempt with backoff; a plain `ToolError` fails immediately, since retrying a deterministic refusal cannot help.
   - **any other exception** — treated as transient and retried with the same exponential backoff used for adapter errors.

Term: Sandbox

A restricted execution environment that limits what a program can access, to contain the effect of something going wrong or being misused. Here, every filesystem tool resolves paths through `ToolContext.resolve()`, which rejects any path that would escape the run's workspace directory (checked via `Path.resolve()` and comparing against the workspace root's parents) — so a model cannot be tricked into reading or writing outside its assigned sandbox folder.

10.3 The seven built-in tools

All defined in `tools/builtin.py`, and all file paths are resolved inside the per-run workspace sandbox:

Tool	What it does	Side effect?	Notes
`read_file`	reads a UTF-8 text file (truncated to 20,000 chars)	no	
`write_file`	writes a UTF-8 text file	**yes**	
`list_files`	recursively lists files under a directory (max 500)	no	
`http_get`	GETs a URL	no	host must be in an explicit allowlist, else `ToolError`; 15s timeout
`csv_update`	`append_row` or `set_cell` on a CSV file	**yes**	out-of-range cell is a `ToolError`
`draft_email`	writes an RFC 5322 `.eml` file to a workspace `outbox/` folder	**yes**	never sends anything over a network; the filename is sanitized from the subject
`calendar_add`	inserts a row into a per-workspace SQLite `calendar.db`	**yes**	
`calendar_list`	reads all rows back, ordered by start time	no	

Term: Allowlist

A list of explicitly permitted values (here, hostnames) where everything not on the list is rejected by default — the opposite of a blocklist, and generally the safer default for anything reachable by an untrusted or semi-autonomous actor like an LLM.

11 Model adapters

Term: Adapter (in this codebase)

A small class implementing one interface (`ModelAdapter.complete(messages, tools) -> ModelTurn`) around a specific way of getting the "next turn" — a real API, or a scripted fake. The rest of the runtime only ever talks to this interface, never to a specific provider's request/response shape directly.

`adapters/base.py` defines the shared `ModelTurn` (text, tool calls, token `Usage`, model name; `is_final` is true exactly when there are no tool calls) and two exception types the runtime specifically catches: `AdapterError` (transport/provider failure, retryable) and `MalformedOutputError` (output that could not be parsed into a turn at all, carrying the raw text for diagnosis).

11.1 `MockModelAdapter` — the offline, deterministic adapter

`adapters/mock.py` plays back a *script*: an ordered list of turns taken straight from a scenario YAML file (Sec. 12), each either `tool_calls`, `final` text, or `malformed` raw garbage. Two separate counters matter here and are easy to conflate (the README singles this out as a real bug the project hit and fixed): `calls` counts *every* invocation of `complete()`, including ones a fault plan turns into an injected error; `script_i` counts only script turns actually *consumed*. Injected faults do not advance `script_i`,

so the script continues exactly where it left off once the fault clears. Usage numbers are fabricated deterministically from message/output size (never a real token count) and always carry `simulated: true`.

11.2 `AnthropicAdapter` and `OpenAIAdapter` — live, but unexercised here

`adapters/anthropic_adapter.py` and `adapters/openai_adapter.py` implement request-building and response-parsing for the Anthropic Messages API (`POST /v1/messages`) and the OpenAI Chat Completions API (`POST /v1/chat/completions`) respectively, each keyed by an environment variable (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`) and using `httpx` for the HTTP transport. Both translate the neutral message list (Sec. 5) into their provider's wire shape, map HTTP ≥500 and other non-200 responses to `AdapterError`, and map an unexpected response shape (e.g. a missing field) to `MalformedOutputError`.

Honest scope note (verbatim from the project's own README)

"The Anthropic and OpenAI adapters are implemented against their current APIs and covered by offline contract tests (request building, response parsing, error mapping via injected HTTP transports), but no live API calls were made here because no API keys were available. Everything else, including the full test suite, the CLI and the web UI, runs and was verified entirely offline."

Term: Contract test

A test that checks a piece of code produces the exact request shape a real API expects, and correctly parses the exact response shape that API returns — without actually calling the real API. It verifies the code's side of the "contract" between the two systems, using a fake/injected transport in place of the network. `tests/test_adapters.py` does this for both live adapters by injecting a fake `httpx.Client`.

12 Agent definitions and example scenarios

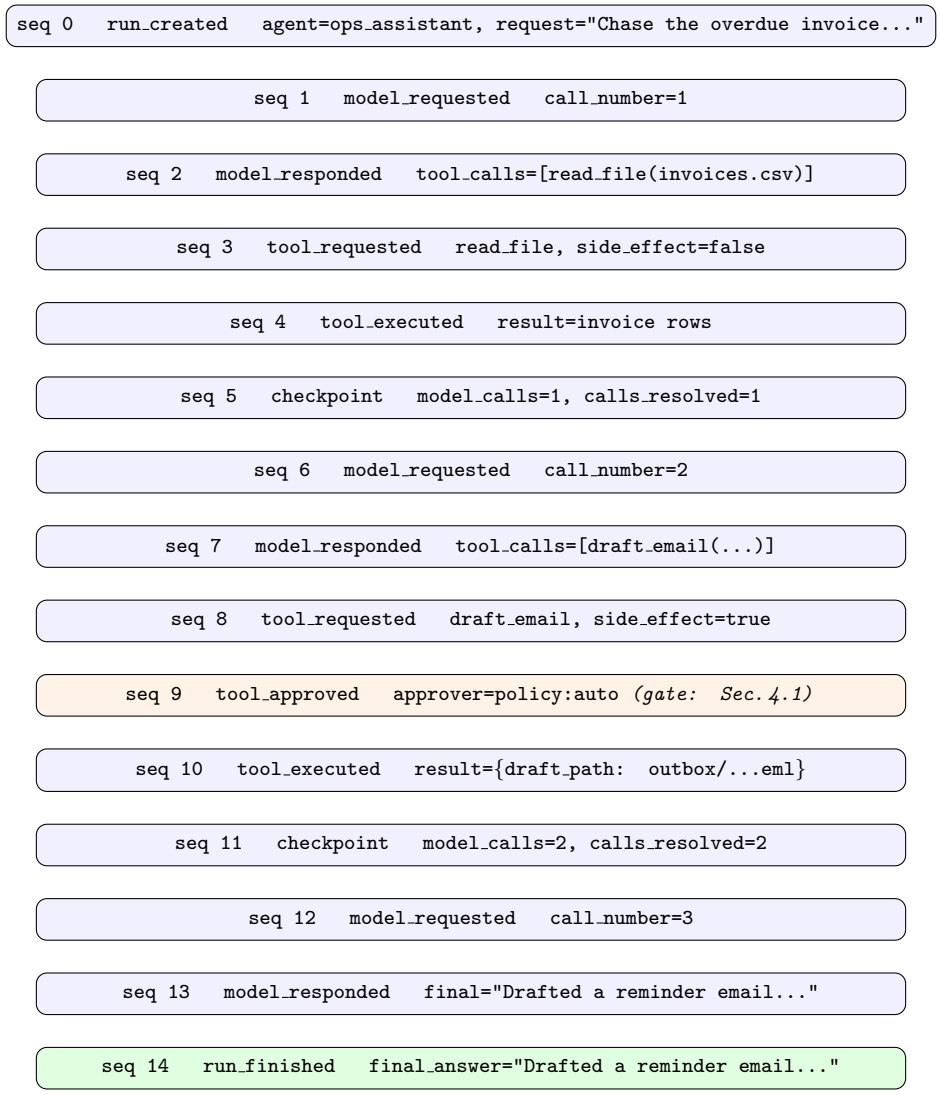
`agents.py` defines three named agents, each a fixed system prompt plus a fixed, restricted subset of tools it may call:

Agent	Role and allowed tools
`ops_assistant`	Chases overdue invoices, drafts (never sends) reminder emails, keeps an invoice CSV current. Tools: `read_file`, `list_files`, `csv_update`, `draft_email`, `calendar_add`, `calendar_list`.
`data_entry`	Transcribes records into CSV sheets, verifying by reading back what it wrote. Tools: `read_file`, `list_files`, `write_file`, `csv_update`.
`research_summarizer`	Fetches allowlisted pages and local notes, writes a summary with sources. Tools: `read_file`, `list_files`, `http_get`, `write_file`.

The repository ships 24 scenario YAML files under `scenarios/`, split evenly across three subfolders matching the three agents: 8 in `data_entry/`, 8 in `ops_assistant/`, 8 in `research/` (counted directly from the files present in the `scenarios/` directory tree).

12.1 A concrete run, end to end

The scenario `scenarios/ops_assistant/chase_overdue_acme.yaml` is a small, representative example: the request is "Chase the overdue invoice from Acme Corp with a reminder email," and behavior `v1` scripts exactly three model turns (read the invoice file, draft the reminder email, give a final answer). Running it through `agentrun run` produces this event sequence (traced from `runtime.py` step by step for this exact script):



With the CLI's default auto-approve policy, `tool_approved` happens automatically at seq 9; run without `--auto-approve` and no policy at all, the loop would instead stop and return `waiting_approval` right before seq 9, and a human would run `agentrun approvals approve <run-id> call-2-0` to produce exactly that event and let the loop continue — the same downstream events (10 through 14) follow either way, because they are computed from the log, not from which code path triggered the approval.

13 Scenario regression testing

Term: Regression test

A test that exists specifically to catch a case where something that used to work now behaves differently, typically run after a change (here: after a model version or prompt changes) to prove nothing that used to pass now fails.

Each scenario YAML (`scenarios/loader.py`, validated against a strict JSON schema) has: a `name`, an `agent`, a `request`, optional `workspace_files` (fixture files created before the run), one or more named `behaviors` (each a full script for `MockModelAdapter`, Sec. 8.1), optional `approvals` (`auto`, or a `deny` list with a `reason`), optional `faults` (Sec. 9), and an `expect` block of assertions.

13.1 Assertions

`scenarios/runner.py`'s `check_expectations()` evaluates every assertion present (never short-circuiting on the first failure, so a failing scenario reports everything wrong at once):

Assertion key	Checks
`status`	final run status equals `finished`/`failed`/`waiting_approval`
`tools_executed`	the *exact*, ordered list of tools that executed successfully
`tools_executed_contains`	these tools ran somewhere (order/extras ignored)
`tools_not_executed`	these tools never ran
`denied_tools`	these tools were denied at the approval gate
`final_contains` / `final_not_contains`	case-insensitive substring checks on the final answer
`failure_cause_contains`	substring of the recorded failure cause
`files_exist`	workspace-relative paths that must exist afterward
`min_events`	minimum total event count
`no_side_effect_before_approval`	every executed side-effecting call had a `tool_approved` event *before* its `tool_executed` event, by sequence number

13.2 Running and comparing versions

`agentrun test scenarios/ --behavior v1 --compare v2 --html report.html` runs every scenario in the pack against behavior `v1`, then again against `v2`, and diffs them. The diff mechanism (`scenarios/runner.event_signature()`) reduces each event to a compact `"type:tool-or-call-id"` string (e.g. `tool_requested:read_file`); two scenarios' signature lists are compared element-by-element, and the first index where they differ is reported as the first point of behavioral divergence — e.g. "expected `tool_requested:read_file`, got `tool_requested:draft_email` at event 3." No general-purpose diff library is used; a list of short strings compared position-by-position is sufficient (`scenarios/report.py`'s `diff_results()`). `terminal_report`, `terminal_diff` and `html_report` render this to the console and to a static HTML file respectively; the HTML report is generated by hand-built string templates, not a templating library.

14 The command-line interface

`cli.py` exposes a single `agentrun` entry point (registered via `pyproject.toml`'s `[project.scripts]`) built with **Click**, a Python library for building command-line tools out of decorated functions.

Command	What it does
`agentrun run --scenario ...`	starts a mock-adapter run scripted by a scenario file; `--auto-approve` skips the human gate
`agentrun resume <run-id>`	rebuilds the exact runtime for an existing run from its `run_created` metadata and drives it forward (Sec. 15)
`agentrun replay <run-id> [--until N]`	deterministic replay (Sec. 7); exits nonzero if not byte-identical
`agentrun runs list / show`	lists all runs with status/cost, or prints one run's full canonical event log
`agentrun approvals list / approve / deny`	manage the human approval queue
`agentrun test <pack> [--behavior --compare --html]`	runs the scenario regression suite (Sec. 12)
`agentrun serve [--host --port]`	serves the FastAPI web UI (Sec. 14) via `uvicorn`
`agentrun agents`	lists the built-in agent definitions and their tools

Every command takes a `--db` option (default `agentrun.db`, overridable by the `AGENTRUN_DB` environment variable) pointing at the SQLite file to use.

15 The web UI

`web/app.py` builds a small **FastAPI** application (a Python web framework for building HTTP APIs, here used to serve plain HTML pages instead) with three routes:

- `GET /` — a table of every run: id, agent, status badge, event count, cost.
- `GET /runs/{run_id}` — one run's status, cost/latency summary, any *pending approvals* rendered as a human-readable **diff-style preview** of what the action would change (Sec. 14.1), and the full event timeline.
- `POST /runs/{run_id}/calls/{call_id}/approve` and `/deny` — record the decision, then call `resume_run()` (Sec. 15) *synchronously, inside the HTTP request handler*, before redirecting back to the run page.

Known limitation (from the README, stated by the project itself)

"Approvals via the web UI resume the run synchronously inside the request handler. A slow tool blocks the HTTP worker for its whole timeout. A real deployment needs a worker process consuming a resume queue, with the UI only appending decision events."

15.1 Diff-style previews

`web/preview.py`'s `preview(tool, args, workspace)` renders, for a *pending* side-effecting call, what it would change, before a human approves it: for `write_file` and `csv_update` it computes a real unified diff (Python's `difflib.unified_diff`) between the current file content and what the call would produce; for `draft_email` and `calendar_add` (which have no natural "before" state to diff against) it renders a plain `+`-prefixed summary of the new content instead.

Term: Unified diff

A standard, compact way to show the difference between two versions of a text file: unchanged lines shown plainly, removed lines prefixed `-`, added lines prefixed `+`. The format produced by `git`

`diff` and Unix `diff -u`.

16 Bootstrap: constructing runtimes for new and existing runs

`bootstrap.py` is the module the CLI and web UI both call into so that "build me a working `Runtime` for this run" is written exactly once. Its key insight, stated in its own docstring: *run metadata (workspace path, adapter kind, scenario file, behavior version) lives in the `run_created` event, so any process with the database can rebuild the exact runtime needed to continue a run* — no separate session state has to survive a process restart.

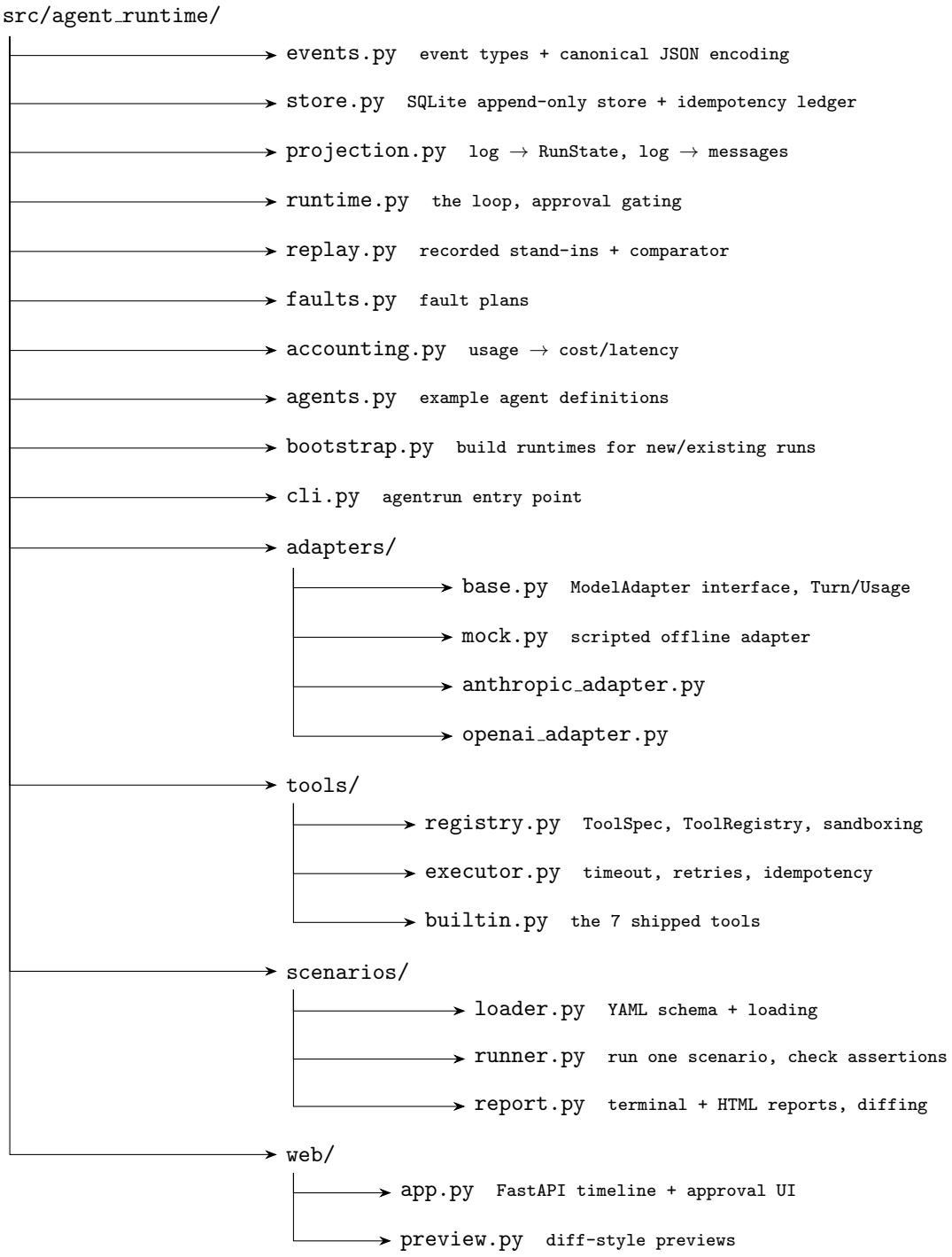
`resume_run(store, run_id, approval_policy)`:

1. Projects the log; if the run is already terminal, returns immediately.
2. Counts how many `model_responded` events were *not* malformed — this is exactly how many script turns the mock adapter has consumed, so the rebuilt `MockModelAdapter`'s `script_i` is repositioned to that count.
3. Separately counts every `model_responded` *or* `model_error` event — this is how many times `complete()` was called in total, used to reposition the mock adapter's `calls` counter so fault matching (`at_call`, Sec. 9) stays aligned after a resume.
4. Calls `Runtime.drive(run_id)`, which picks up exactly where the fixed point iteration (Sec. 4) left off.

Known limitation (from the README, stated by the project itself)

"Resume repositions the mock adapter by counting events. It is correct for the states the tests cover, but a run resumed in the middle of an injected model-fault sequence could realign fault triggers off by one. The honest fix is recording the adapter cursor in a checkpoint payload instead of inferring it."

17 Package map



Alongside `src/`, the repository root has `scenarios/` (24 YAML fixture files, three subfolders by agent), `tests/` (13 test modules, 1,357 lines, 93 test functions — counted directly from the files present), `docs/` (`ARCHITECTURE.md`, `WRITING_SCENARIOS.md`), plus the standard `README.md`, `LICENSE` (MIT), `pyproject.toml`, `requirements.txt` and `.env.example`.

18 Dependencies

From `pyproject.toml` (runtime) and the `dev` extra:

Package	Role in this project
`click`	builds the `agentrun` CLI
`PyYAML`	parses scenario `.yaml` files
`jsonschema`	validates tool arguments and scenario file structure against JSON Schema
`httpx`	the HTTP client used by the live adapters and the `http_get` tool
`fastapi` + `uvicorn`	the web UI and its ASGI server
`python-multipart`	required by FastAPI to parse the deny form's `reason` field (see the README's account of the one real bug this caused, below)
`pytest` (dev only)	the test suite

A real bug the project's own history records

From the README: "The first full-suite run failed with 5 errors in the web tests only: FastAPI's `Form` parsing needs `python-multipart`, which nothing else imports, so every non-web test passed and the gap only surfaced when the deny-with-reason endpoint was exercised through `TestClient`. Pinned it as a real dependency rather than a test extra since the deny form is a runtime feature." This is reflected in `pyproject.toml`, where `python-multipart` is listed as a core dependency, not under `dev`.

19 Testing

The test suite (`tests/`, run with `pytest`) has 93 test functions across 13 files (counted directly: `test_accounting.py`, `test_adapters.py`, `test_cli.py`, `test_executor.py`, `test_faults.py`, `test_replay.py`, `test_resume.py`, `test_runtime.py`, `test_scenarios.py`, `test_store.py`, `test_tools.py`, `test_web.py`, plus shared fixtures in `conftest.py`), matching the count the README states ("93 tests, all offline"). `conftest.py` provides a `make_runtime()` helper that wires a fresh in-memory-style SQLite store, a sandbox workspace, the mock adapter and an auto-approve policy with a single call, used throughout the other test files. Every test runs offline: no test in this repository makes a live network call to Anthropic or OpenAI.

20 What the project's own retrospective says

The README includes a "What I learned" and "What I'd do differently" section written by the project's author. Reproduced here verbatim (not paraphrased, so nothing is lost or oversold) because it is part of the project's own honest account of its design and limits:

What was learned

- Deriving all state from a projection collapsed the feature count: resume after crash, continue after approval, and a plain next loop iteration became literally the same function, so the recovery tests exercise the normal path rather than a bespoke one.
- Determinism is a budget spent field by field: any payload value that is not a pure function of the log (a timestamp, a real-sleep-dependent attempt count, a random id) is a future replay divergence.

- Separating expected tool failures (`ToolError`) from unexpected ones buys a retry policy for free: deterministic refusals fail fast, transient exceptions and timeouts consume the retry budget with backoff.
- `concurrent.futures` timeouts abandon the worker thread rather than kill it — a timeout answers "how long do I wait," not "how do I stop the tool." Documented rather than papered over.
- A per-event `"type:detail"` signature is enough for useful regression diffs, with no diffing library needed.

What the author would do differently

- The web UI's synchronous approval-resume should become a worker process consuming a resume queue.
- Resume's event-counting approach to repositioning the mock adapter is correct for tested states but could misalign fault triggers in an edge case; a recorded adapter cursor in the checkpoint payload would be the honest fix.
- Event payloads carry no schema version; replay currently detects divergence as a tripwire, not a migration story.
- Tool timeouts do not kill the handler thread, so a truly hung tool leaks a thread per attempt; subprocess isolation would fix this and give tools their own memory space.
- The in-house neutral message format ignores streaming and provider-specific parallel tool-call semantics; the accounting price table is hardcoded and will drift.

21 Glossary

Agent A program that uses an LLM in a loop to decide and take actions via tools.

Adapter A class implementing a fixed interface around a specific model API or fake.

Allowlist A set of explicitly permitted values; everything else is rejected.

Approval gate The point in the loop where a side-effecting tool call pauses for a human decision.

Backoff Waiting longer between each successive retry.

Canonical encoding A deterministic serialization where equal data always produces an identical string.

Checkpoint A periodic event recording loop progress, for readability, not correctness.

Contract test A test verifying request/response shape against a real API's spec, without calling the real API.

Digest / hash A short fixed-length fingerprint of a larger piece of data.

Event sourcing Storing every state change as an immutable ordered log instead of overwriting current state.

Fixed-point iteration A loop that recomputes its next step purely from its own last output.

Idempotency Doing an operation twice has the same effect as doing it once.

JSON Schema A specification language describing the required shape of JSON data.

LLM	Large language model: a machine learning model that generates text (or structured output) from a prompt.
Primary key	A column set guaranteed unique per row, used to enforce no-duplicate writes.
Projection	Computing derived state from an event log.
Regression test	A test proving something that used to work still works after a change.
Replay	Re-driving logic from only recorded data, with no live calls, and checking the result matches.
Sandbox	A restricted environment limiting what a program can access.
Side effect	An action with a lasting, observable consequence outside the program.
Unified diff	A standard compact text format for showing line-level differences between two versions of a file.